



Manual Técnico



Versión: Junio 11, 2026

Índice

Introducción y Propósito del Sistema	2
Arquitectura General del Sistema	2
Flujo de Comunicación	2
Justificación de Diseño	3
Pila Tecnológica (Tech Stack)	3
Frontend	3
Backend	3
Descripción de Módulos del Backend	4
Endpoints y Rutas del Sistema	5
APIs REST del Backend	5
Aplicaciones Embebidas Dash	5
Configuración de Despliegue	6
Backend — Render	6
Frontend — Vercel	6
Requisitos Previos	7
Configuración del Backend	7
1. Clonamos el repositorio	7
2. Crear y activar entorno virtual	8
3. Instalar dependencias de Python	8
4. Iniciar el servidor Flask en modo desarrollo	9
Configuración del Frontend	9
1. Navegar al directorio del frontend	9
2. Instalar dependencias de Node	9
3. Iniciar el servidor de desarrollo Vite	10
Resumen de Referencia	10

Introducción y Propósito del Sistema

La plataforma de analítica fue diseñada para procesar y visualizar datos operativos de distribuidores de la maquinaria de CNHMX. El sistema integra datos variados (mantenimientos, unidades, población, ruteo) en un único sistentorno de negocio, con dashboards interactivos y modelos estadísticos que permiten tomar decisiones basadas en datos en tiempo real.

El proyecto está orientado a tres áreas clave de análisis:

- **Riesgo Operativo:** Identificación de unidades en riesgo mediante modelos de clustering (K-Means).
- **Fuga de Servicios:** Detección de patrones de abandono en contratos de servicio y mantenimiento.
- **Monetización y Potencial:** Visualización del potencial de ingresos por distribuidor y región.

Este manual está dirigido a ingenieros de software, DevOps y administradores de sistemas responsables del mantenimiento y evolución del proyecto.

Arquitectura General del Sistema

El sistema adopta un patrón de Frontend/Backend con una integración híbrida de dashboards Dash. Esta arquitectura permite que el frontend de React actúe como una Single Page Application (SPA) independiente que consume APIs REST, mientras que el backend Flask concentra toda la lógica de negocio, procesamiento de datos y generación de visualizaciones analíticas complejas.

Flujo de Comunicación

La siguiente tabla describe los canales de comunicación entre los componentes del sistema:

Origen	Destino / Canal	Tipo de Interacción
Navegador (Usuario)	Frontend React (Vite)	SPA servida estáticamente desde Vercel
Frontend React	Backend Flask	Peticiones HTTP/REST (JSON)
Navegador (Usuario)	Backend Flask	Embeds de dashboards Dash via iframe/redirect
Backend Flask	Disco / CSV Cache	Lectura y escritura de datos consolidados

Justificación de Diseño

- **Separación de responsabilidades:** el frontend no contiene lógica de negocio; el backend no gestiona estado de UI.
- **Caché en disco:** los datos procesados se persisten en CSV para evitar reprocesamiento en cada petición.
- **Optimización de memoria:** se evita el uso de formatos pesados (XLSX) en producción para respetar los límites de RAM del servidor.
- **Despliegue continuo:** push a la rama main dispara el despliegue automático en Render y Vercel.

Pila Tecnológica (Tech Stack)

Frontend

Tecnología	Versión / Detalle	Rol en el Sistema
React	19.2.6	Framework principal de UI
TypeScript	6.0.2	Tipado estático para mayor robustez del código
Vite	8.0.12	Compilador y empaquetador ultrarrápido
React Router DOM	7.16.0	Gestión del enrutamiento SPA del cliente
Lucide React	1.17.0	Biblioteca de iconos SVG ligeros
CSS Modules	<i>(Integrado por defecto en Vite)</i>	Estilos encapsulados por componente

Backend

Tecnología	Versión / Detalle	Rol en el Sistema
Flask	Python 3.11	Framework web y servidor de APIs REST
Dash (Plotly)	2.15.0	Motor de dashboards interactivos analíticos
Pandas	2.2.0	Manipulación, fusión y análisis de DataFrames
NumPy	1.26.4	Operaciones numéricas y matemáticas
Gunicorn	21.2.0	Servidor WSGI para producción
openpyxl	3.1.2	Lectura de archivos Excel en la importación

Estructura de Directorios del Proyecto

El proyecto sigue una estructura modular clara que separa el código fuente del frontend, el backend y los datos persistentes:

```
EMAX_CNHMX/
├─ app/                                # Código fuente del Backend Flask
│   ├── app.py                        # Servidor principal y definición de rutas API
│   ├── importador.py                # Módulo de carga, validación y mezcla de datos
│   ├── dashboard.py                 # Lógica de procesamiento de KPIs y negocio
│   ├── riesgo_operativo.py          # Dash/Analytics – Riesgo Operativo (K-Means)
│   ├── fuga_servicios.py            # Dash/Analytics – Fuga de Servicios
│   └─ monetizacion.py               # Dash/Analytics – Monetización y Potencial
├─ data/                              # Almacenamiento local de base de datos
│   ├── ArchivosLimpios/              # Archivos CSV consolidados de producción
│   └─ archivos_compañía/            # Archivos auxiliares corporativos
├─ frontend/                          # Código fuente del Frontend React
│   ├── src/
│   │   ├── presentation/            # Componentes y Páginas (Pages) de la UI
│   │   ├── config.ts                # Parámetros globales de API (URL base)
│   │   └─ App.tsx                   # Configuración de rutas frontend
│   └─ package.json                  # Dependencias y scripts Node
├─ notebooks/                         # Cuadernos Jupyter para experimentación
└─ requirements.txt                   # Dependencias Python del Backend
```

Descripción de Módulos del Backend

Módulo	Responsabilidad Principal
app.py	Punto de entrada del servidor Flask. Registra rutas REST y monta las aplicaciones Dash como sub-aplicaciones.
importador.py	Realiza la recepción de archivos Excel, su validación por esquema, fusión con el CSV histórico y almacenamiento optimizado.
dashboard.py	Calcula y expone los KPIs consolidados: métricas por distribuidor, alertas y resúmenes ejecutivos.
riesgo_operativo.py	Implementa el modelo K-Means sobre los datos de mantenimiento para segmentar unidades por nivel de riesgo operativo.
fuga_servicios.py	Analiza patrones de fuga en los contratos de servicio y genera visualizaciones comparativas.

monetizacion.py	Proyecta el potencial de ingresos y genera gráficos de monetización por distribuidor y región geográfica.
-----------------	---

Endpoints y Rutas del Sistema

APIs REST del Backend

Todas las rutas REST son servidas por el servidor Flask y devuelven respuestas en formato JSON. La URL de la base del backend en producción se configura mediante la variable de entorno VITE_API_BASE_URL ubicada en el frontend.

Método	Ruta	Descripción
GET	/api/status	Retorna 200 OK si el backend está disponible. Usado por el frontend para verificar conectividad antes de renderizar la UI.
POST	/api/upload	Recibe los archivos .xlsx, los clasifica, valida, fusiona con el historial y los convierte en CSV. Retorna un resumen del proceso de importación.
GET	/api/dashboard	Regresa los KPI de servicios totales, alertas y resúmenes agregados por distribuidor.
GET	/api/distribuidores	Regresa el detalle de métricas operativas por distribuidor de las unidades (unidades activas, mantenimientos pendientes, etc.).

Aplicaciones Embebidas Dash

Las siguientes rutas sirven para las aplicaciones Dash directamente desde el backend de Flask. Se acceden desde el frontend o abriendo el dominio del backend en el navegador.

Ruta	Dashboard	Tecnología Analítica
/dash/riesgo/	Riesgo Operativo	Modelo K-Means — segmentación de unidades por nivel de riesgo.
/api/dash/fuga/side/	Fuga de Servicios	Análisis de tendencias de abandono de contratos.
/dash/monetizacion/	Monetización y Potencial	Proyección de ingresos y análisis de potencial por distribuidor.

Configuración de Despliegue

El proyecto está preparado para despliegue continuo (CI/CD) en dos plataformas. Cualquier push a la rama main del repositorio dispara automáticamente el proceso de build y despliegue en ambas plataformas.

Backend — Render

Parámetro	Valor / Configuración
Tipo de Servicio	Web Service (Python 3)
Versión de Python	3.11
Comando de Inicio	gunicorn --bind 0.0.0.0:\$PORT --timeout 120 app.app:app
RAM Disponible	512 MB (Capa Gratuita)
Rama de Despliegue	main
Auto-deploy	Activo — push a main dispara rebuild automático

El parámetro `--timeout 120` es muy importante para permitir que las operaciones de importación de archivos grandes completen su ejecución sin que Gunicorn interrumpa el trabajo por timeout.

Frontend — Vercel

Parámetro	Valor / Configuración
Tipo de Servicio	Static Web App (Vite / Node)
Comando de Build	npm run build
Directorio de Salida	dist/
Variable de Entorno	VITE_API_BASE_URL
Valor de la Variable	https://cnhmx-analisis.onrender.com (URL pública de Render)
Rama de Despliegue	main
Auto-deploy	Activo — push a main dispara rebuild automático

Variables de Entorno Requeridas

VITE_API_BASE_URL — Debe configurarse en el panel de Vercel antes del primer despliegue.

Su valor debe ser el dominio público asignado por Render al backend (ej: <https://cnhmx-analisis.onrender.com>).

Esta variable se inyecta en tiempo de compilación por Vite. Un cambio requiere un nuevo build.

Nunca exponer claves secretas o tokens en variables VITE_* ya que son visibles en el bundle del cliente.

Guía de Desarrollo Local

Requisitos Previos

- Python 3.11 instalado y disponible en PATH.
- Node.js 18+ con npm instalado.
- Git configurado con acceso al repositorio.

Configuración del Backend

1. Clonamos el repositorio

```
PS C:\Users\ximen\Downloads> git clone https://github.com/XimenaCantera/EMAX_CNHMX.git
Cloning into 'EMAX_CNHMX'...
remote: Enumerating objects: 1020, done.
remote: Counting objects: 100% (439/439), done.
remote: Compressing objects: 100% (296/296), done.
remote: Total 1020 (delta 253), reused 299 (delta 139), pack-reused 581 (from 1)
Receiving objects: 100% (1020/1020), 28.37 MiB | 4.53 MiB/s, done.
Resolving deltas: 100% (550/550), done.
warning: the following paths have collided (e.g. case-sensitive paths
on a case-insensitive filesystem) and only one from the same
colliding group is in the working tree:

    'frontend/src/presentation/components/common/Cargador.tsx'
    'frontend/src/presentation/components/common/cargador.tsx'
PS C:\Users\ximen\Downloads>
```



```

● PS C:\Users\ximen\Downloads> cd EMAX_CNHMX
● PS C:\Users\ximen\Downloads\EMAX_CNHMX> ls

Directorio: C:\Users\ximen\Downloads\EMAX_CNHMX

Mode                LastWriteTime         Length Name
----                -
d-----         12/06/2026   09:12 a. m.      app
d-----         12/06/2026   09:12 a. m.      data
d-----         12/06/2026   09:12 a. m.      frontend
d-----         12/06/2026   09:12 a. m.      notebooks
-a-----         12/06/2026   09:12 a. m.      875 .gitignore
-a-----         12/06/2026   09:12 a. m.      5306 Prueba_Mantenimientos_Adicionales.xlsx
-a-----         12/06/2026   09:12 a. m.      5387 README.md
-a-----         12/06/2026   09:12 a. m.      342 requirements.txt
-a-----         12/06/2026   09:12 a. m.      349 vercel.json

○ PS C:\Users\ximen\Downloads\EMAX_CNHMX>

```

2. Crear y activar entorno virtual

```

● PS C:\Users\ximen\Downloads\EMAX_CNHMX> python -m venv venv
● PS C:\Users\ximen\Downloads\EMAX_CNHMX> venv\Scripts\activate
❖ (venv) PS C:\Users\ximen\Downloads\EMAX_CNHMX>

```

3. Instalar dependencias de Python

```

○ (venv) PS C:\Users\ximen\Downloads\EMAX_CNHMX> pip install -r requirements.txt
Collecting Flask==3.0.2 (from -r requirements.txt (line 4))
  Downloading flask-3.0.2-py3-none-any.whl (101 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 101.3/101.3 kB 6.1 MB/s eta 0:00:00
Collecting Flask-Cors==4.0.0 (from -r requirements.txt (line 5))
  Downloading Flask_Cors-4.0.0-py2.py3-none-any.whl (14 kB)
Collecting pandas==2.2.0 (from -r requirements.txt (line 6))
  Downloading pandas-2.2.0-cp311-cp311-win_amd64.whl (11.6 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.6/11.6 MB 17.7 MB/s eta 0:00:00
Collecting numpy==1.26.4 (from -r requirements.txt (line 7))
  Using cached numpy-1.26.4-cp311-cp311-win_amd64.whl (15.8 MB)
Collecting openpyxl==3.1.2 (from -r requirements.txt (line 8))
  Downloading openpyxl-3.1.2-py2.py3-none-any.whl (249 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 250.0/250.0 kB 15.0 MB/s eta 0:00:00
Collecting dash==2.15.0 (from -r requirements.txt (line 9))
  Downloading dash-2.15.0-py3-none-any.whl (10.2 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.2/10.2 MB 11.9 MB/s eta 0:00:00
Collecting plotly==5.18.0 (from -r requirements.txt (line 10))

```

4. Iniciar el servidor Flask en modo desarrollo

```
PS C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX> python app/app.py
[RIESGO] Procesando datos desde C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX\data\ArchivosLimpios\new_mantenimie
ntos.xlsx...
[RIESGO] Datos procesados en 1.55 segundos. Cacheando resultados...
Total de unidades Crítico + Alto graficadas: 2445
Top estados recomendados: ['Jalisco', 'Puebla', 'Guanajuato']
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serve
r instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.25.69.65:5000
Press CTRL+C to quit
* Restarting with stat
```

Configuración del Frontend

1. Navegar al directorio del frontend

```
PS C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX> cd frontend
PS C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX\frontend>
```

2. Instalar dependencias de Node

```
PS C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX\frontend> npm install

up to date, audited 158 packages in 2s

43 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX\frontend>
```

3. Iniciar el servidor de desarrollo Vite

```
PS C:\Users\ximen\Desktop\cnh_mx\EMAX_CNHMX\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.0.16 ready in 538 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Resumen de Referencia

Componente	URL / Comando de Referencia
Backend (Producción)	https://cnhmx-analisis.onrender.com
Frontend (Producción)	Dominio asignado por Vercel
Dashboard Riesgo	/dash/riesgo/
Dashboard Fuga	/api/dash/fuga/side/
Dashboard Monetización	/dash/monetizacion/
Health Check	GET /api/status
Importar Datos	POST /api/upload
Iniciar Backend Local	python app.py (desde /app)
Iniciar Frontend Local	npm run dev (desde /frontend)
Build Frontend Prod.	npm run build (desde /frontend)
Inicio Producción Backend	gunicorn --bind 0.0.0.0:\$PORT --timeout 120 app.app:app